TECHNICAL PRODUCT DOCUMENTATION

Medical Record Digitisation Platform

An on-premise AI system that turns scanned and photographed hospital records, both printed and handwritten, into checked, searchable digital text and structured patient data.

1 Executive summary

What the product is, what it delivers today, and where it stands.

The platform runs as one web application on the hospital's on-premise server. Users sign in and upload a scanned PDF or a photo. The system turns every page into editable digital text and pulls out the clinical fields: patient details, diagnosis, medicines, doses and lab values. Each document is filed under the patient's record by UHID.

2 Product overview

Who it is for and what it does.

2.1 Key capabilities

Capability	Description
Mixed-document reading	Printed, handwritten and mixed pages in one upload (PDF or image).
Automatic orientation	Sideways or upside-down pages and phone photos are turned upright before reading.
Smart routing	Fully printed pages go to OCR on the CPU; handwriting goes to the AI vision model.
Layout-preserving output	Tables, checklists, headers and diagrams are kept as they appear on the page.
Diagram and chart handling	Drawings are cut out and embedded. CBC "fishbone" diagrams get a dedicated reading.
Red-word review	Uncertain handwritten words are highlighted, with a count of words to review per page and per document.
Structured fields	Patient identifiers, diagnosis, medicines (with dose and frequency), lab values, dates, doctors.
Patient records	Documents linked by UHID; values routed into typed tables (encounters, reports, conditions, measurements, medications, genomic findings, services).
Export	Corrected document downloadable as PDF or Word.

2.2 User journey

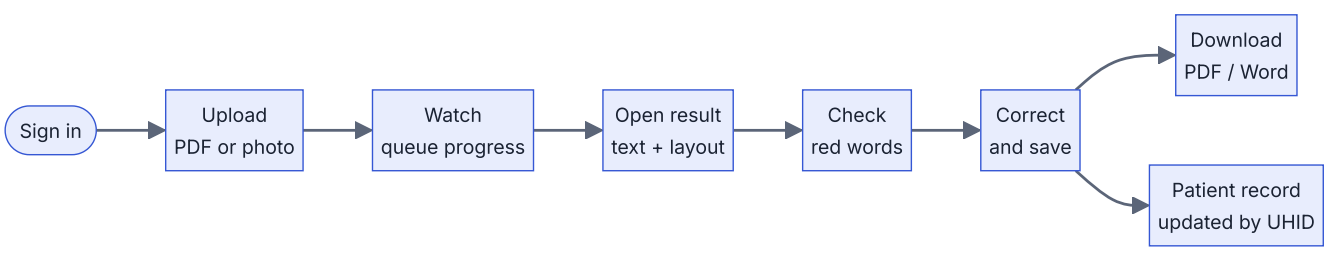


Figure 1: The user journey from upload to a saved, corrected record.

3 System architecture

Four layers in four containers on one on-premise server.

The platform is deployed with Docker Compose. It has four layers:

- the **presentation layer** (the browser application);
- the **application layer** (a Python backend on the CPU that runs all processing logic, including printed-text OCR);
- the **AI inference layer** (the offline vision-language model on the GPU);
- the **data layer** (databases and file storage).

The application and the model talk over an HTTP interface. Either side can be replaced without rewriting the other.

3.1 Architecture diagram

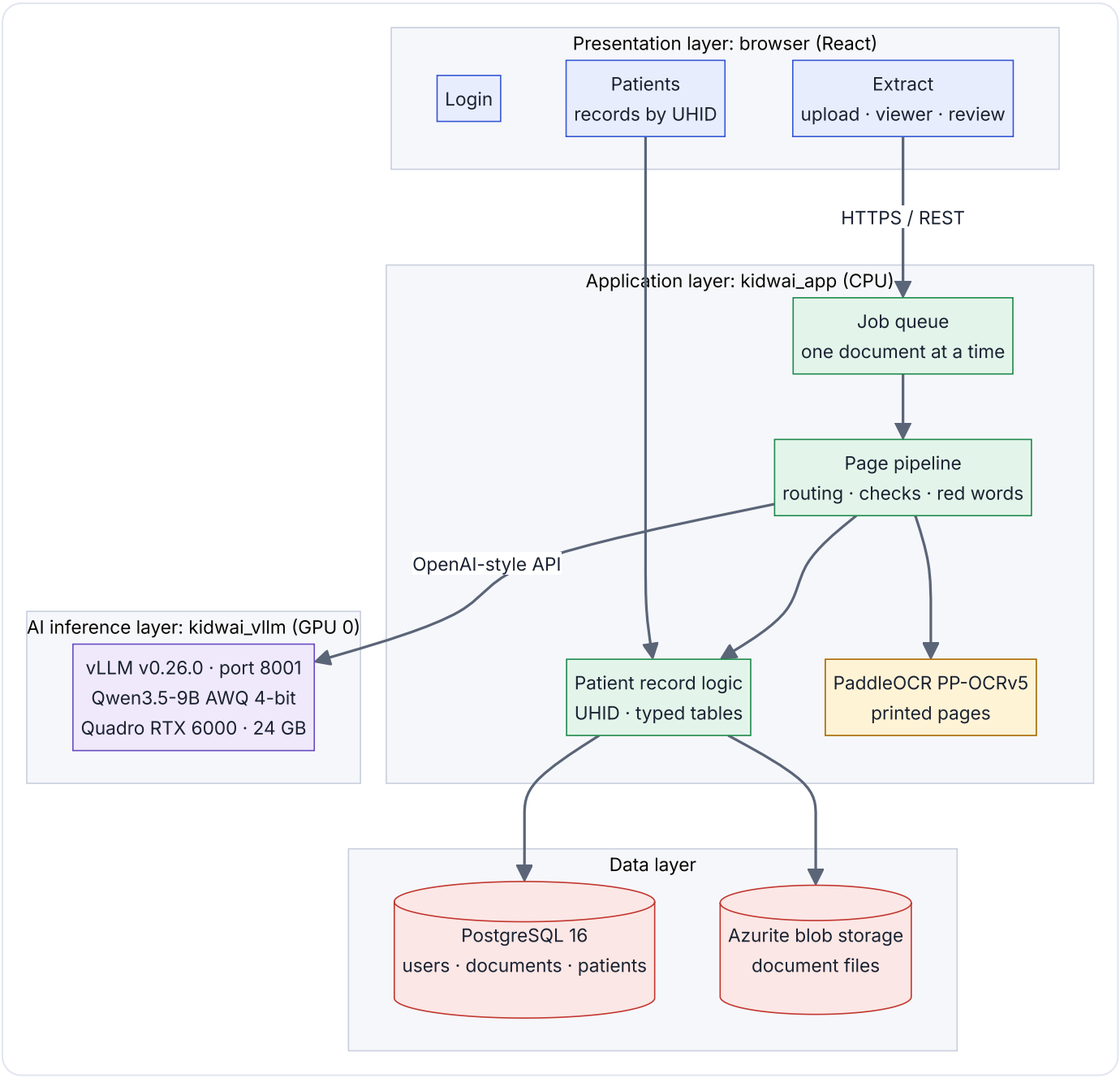


Figure 2: System architecture: layers, containers and connections.

3.2 Containers

Container	Image / technology	Hardware	Responsibility
`kidwai_app`	Python (FastAPI) backend + React front end; PaddleOCR	CPU only	Web UI and API, job queue, page routing, OCR, all checking rules, field merging, patient filing
`kidwai_vllm`	`vllm/vllm-openai:v0.26.0`, model `QuantTrio/Qwen3.5-9B-AWQ`	GPU 0	All vision-language model calls (probes, reads, audits, fields)
`kidwai_users_db`	`postgres:16`	CPU, disk	Users, documents, patients, typed clinical tables, mapping decisions
`kidwai_azurite`	Azurite (Azure Blob emulator)	CPU, disk	Original document files and corrected PDFs

3.3 Technology stack

Area	Technology
Front end	React + TypeScript (Vite build); in-browser PDF and Word export
Back end	Python, FastAPI, thread pools for parallel page work
AI model	Qwen3.5-9B vision-language model, run offline; AWQ 4-bit quantisation
Model serving	vLLM v0.26.0 (OpenAI-compatible API)
Printed OCR	PaddleOCR PP-OCRv5 (server detector, English mobile recogniser)
Databases	PostgreSQL 16 (Alembic migrations)
File storage	Azure Blob API, served by Azurite on-site
Deployment	Docker Compose on a single Linux server with NVIDIA GPU

4 End-to-end processing flow

What happens from the moment a document is uploaded.

Each upload becomes a **job** in a queue, and documents are processed one at a time. If the same file is already queued or running, the upload joins that job instead of starting a second one. If a reviewer has saved a correction for this file, the corrected result is loaded and no AI work is repeated. Otherwise the document is processed again from scratch, even if it was processed before, and the new result replaces the earlier one in storage. Setting `REUSE_CORRECTED_RESULT=0` turns off reuse of corrections, so every upload gets a fresh run.

For a new document, the pages are rendered and turned upright, and the handwriting probe runs on all pages at once. Then two things run in parallel: the **page pipeline**, which reads every page, and **field extraction**, which pulls out the clinical facts for the whole document. When both finish, the result is saved, filed under the patient and shown for review.

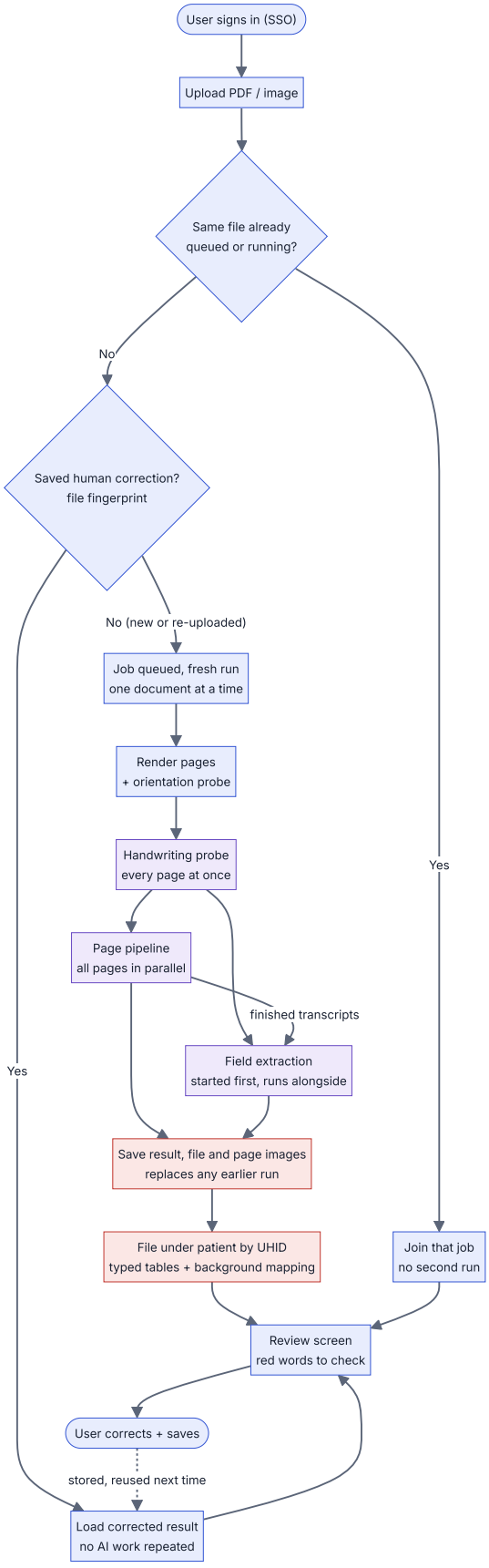


Figure 3: End-to-end processing flow.

#	Stage	Runs on	What it does
1	Sign-in	App + SSO	Identity verified through the organisation's SSO
2	Upload and queue	App	File fingerprinted; a re-upload is run fresh and replaces the earlier result, unless a saved correction exists (then that is loaded)
3	Render and orient	App + GPU	PDF split into page images; each turned upright
4	Handwriting probe	GPU	Yes/no per page; decides the route (section 5)
5	Page pipeline	CPU or GPU	OCR for printed pages; AI reading and checks for handwriting (section 6)
6	Field extraction	GPU	Clinical fields for the whole document, in parallel (section 11)
7	Save	Postgres, Azurite	Result, original file and page images stored; a later run of the same file overwrites them
8	Patient filing	App (+ GPU in background)	Linked by UHID; values routed into typed tables
9	Human review	Browser	User confirms red words; corrections saved

5 Probes and page routing

Small, fast questions asked of the model before the expensive reading starts.

A **probe** is a short, targeted model call. It looks at a downscaled page image and answers one narrow question in a few tokens, for example "is this page upright?" or "does it contain handwriting?". Probes are cheap, but they decide how much expensive work a page needs.

5.1 Orientation probe

Phone photos and scans often arrive sideways or upside down, and a model reading a sideways page produces garbage. Each page is checked before anything else:

- **Portrait pages** are assumed upright. They are flipped 180° only if the model confirms that the original is *not* upright *and* that the flipped version *is*, so a flaky answer can't break a good page.
- **Landscape pages** are usually sideways photos. The system tries 90° clockwise, then 90° anticlockwise, and keeps the first one the model confirms. If neither is confirmed, the page stays as it is, because it is a genuine landscape document.

5.2 Handwriting - OCR probe (routing)

The handwriting probe classifies each page as **none**, **minor** or **substantial** handwriting. It runs on **all pages at once**, before any page reading starts. When it ran page by page, one late page's probe waited 142 s behind other pages' long reads.

- **None or minor** (for example a printed report with only a signature): the page is a candidate for the **printed route**, PaddleOCR on the CPU.
- **Substantial** or **unknown**: the page goes to the **vision model route**.

The same verdict is reused later. On a page called printed, no word is marked red, and the expensive second read and audit are skipped.

6 Page pipeline

How each page is turned into text, layout and red words.

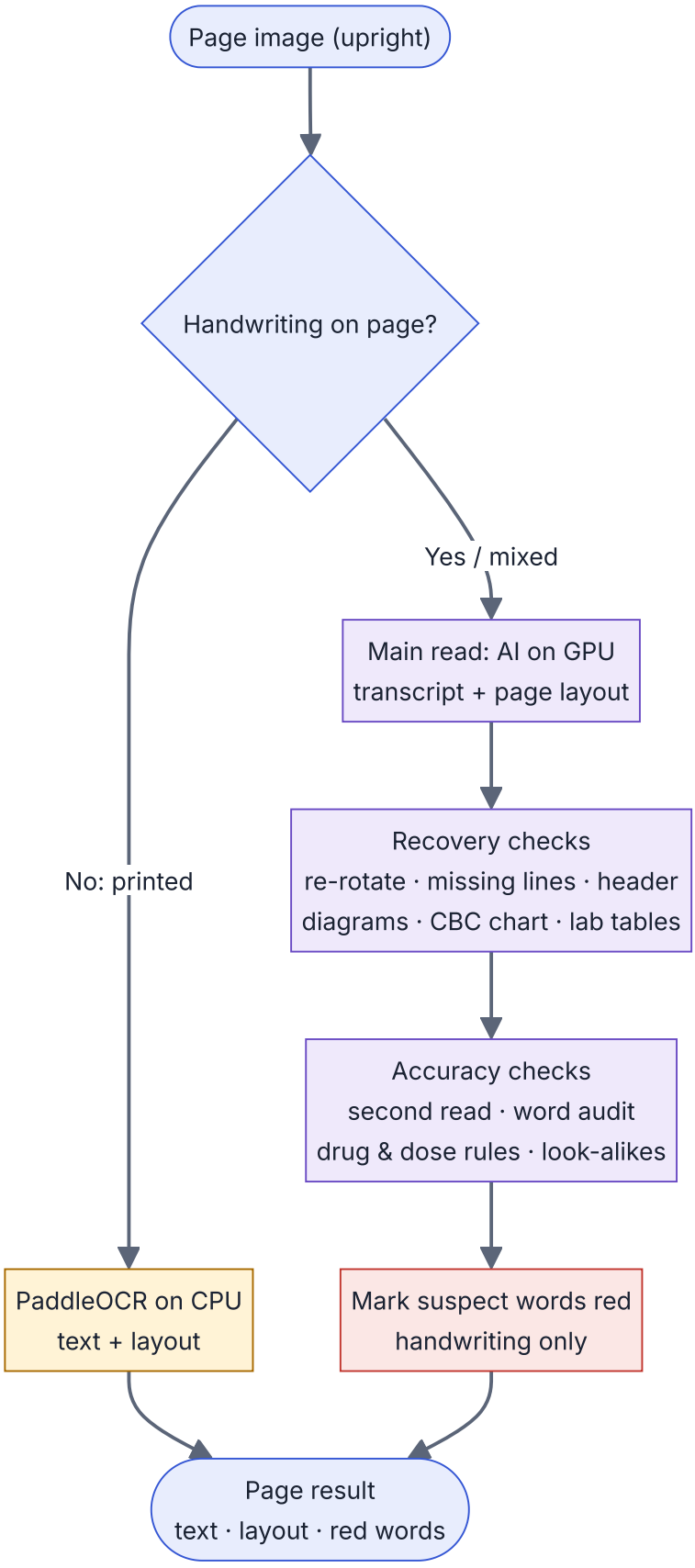


Figure 4: The per-page pipeline.

6.1 Stage explorer

Select a stage to see what it does.

Printed route

PaddleOCR PP-OCRv5 runs on the CPU with the *server* text detector and the English *mobile* recogniser, the combination that tested best. Table rows from phone photos are rebuilt from the positions of the text boxes; otherwise they come out scrambled.

Main read

One combined vision call per page produces both the **plain transcript** and the **layout HTML** (headings, tables, checklists, handwriting spans), so the image is paid for once and both views agree. Handwritten text is wrapped in `hw` spans, which later decide where red is allowed.

The model marks words it doubts with an internal "(?)" marker. The marker is hidden on screen and in exports, and becomes a red highlight.

Recovery

- **Sparse-page rescue:** an empty-looking page is retried at other rotations.
- **Missed-line verification:** recovers skipped lines.
- **Header recovery:** guarantees the top-of-page IDs and patient box.
- **Diagrams:** detected, cut from the image and embedded where the model marked them.
- **CBC fishbone:** a dedicated reading of the blood-count cross diagram.
- **Lab tables:** if text is clearly tabular but the layout has no table, a real table is built in code.
- **Cleanup:** shorthand repair, translation of stray non-English script, checklist de-duplication.

Accuracy checks

Only on pages with handwriting. A **second independent read**, a **word audit** call and **rule-based checks** (unknown drugs, implausible doses, look-alike drug names) each raise flags.

Output

Each page returns its text, its layout HTML with red-word spans and its probe verdict. The UI counts "words to review" only on pages the probe did not call printed. The page image is kept, so a later human correction can be cropped from the exact image the model read.

7 Inference approach

How the AI model is served and called.

All model reading runs on one locally hosted vision-language model, **Qwen3.5-9B**, compressed to 4-bit (AWQ) and served by **vLLM v0.26.0** on GPU 0. The 4-bit model fits the 24 GB card with room left for many requests at once. Nothing leaves the hospital network.

7.1 Design principles

1 · Use the GPU only where it adds value

Printed pages are read by OCR on the CPU. The model reads only handwritten and mixed pages.

2 · Run everything in parallel

All pages are read at once. Field extraction is submitted *first* and runs alongside the pages, not after them.

3 · Read handwriting twice

A main read plus an independent second read, compared word by word, plus an audit and rules. The model is never trusted blindly.

4 · Every call is bounded

Images are capped in size, replies are capped in length, and a loop guard stops runaway output. One bad page can't stall a document.

7.2 How a document uses the model

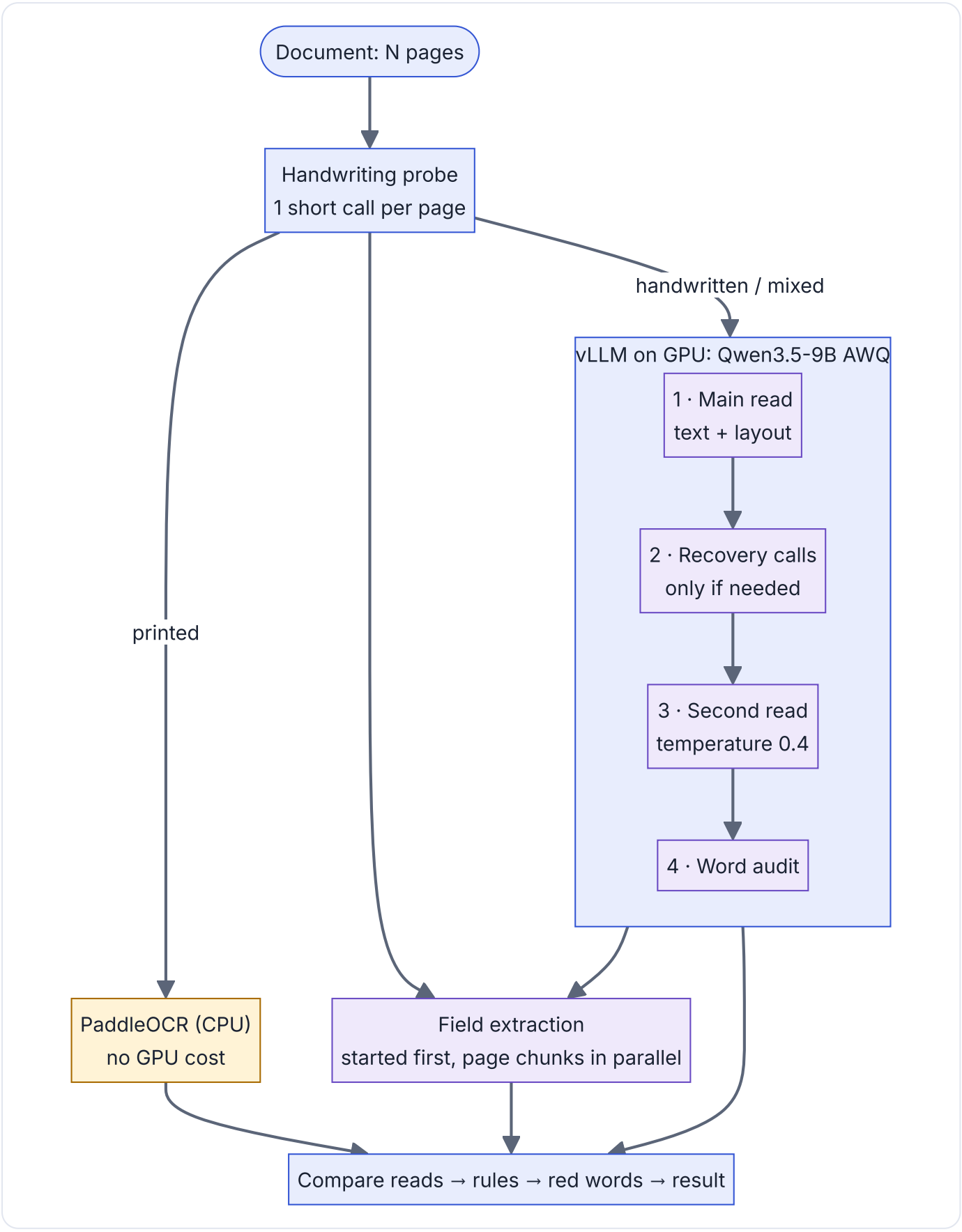


Figure 5: Model calls for one document.

7.3 Serving configuration

Setting	Value	Why
Engine	vLLM v0.26.0, OpenAI-compatible API, port 8001 (localhost only)	Many requests at once; faster batched output than llama-server (~76 vs ~57 tokens/s)
Model	`QuantTrio/Qwen3.5-9B-AWQ`, served as `qwen3.5`	Best accuracy and speed balance that runs on this GPU
Precision	fp16 (`--dtype half`)	Turing has no bf16
Requests in flight	Up to 32 (`--max-num-seqs`)	Keeps the GPU fully busy
Prompt tokens per step	2,048 (`--max-num-batched-tokens`)	Stable; 16,384 crashed the engine
Context length	40,960 tokens	Fits a ~9,300-token fields prompt plus up to 28,000 output tokens
Image size cap	4.2 MP per page image; 1 MP for diagram calls	Full-size images made prompts ~35% longer and doubled read time
Images per request	Up to 4	Field extraction sends pages in chunks
Prefix caching	Off	Deadlocks with Qwen3.5 on v0.26
GPU memory share	90%	Leaves room for the other ~1 GB process on GPU 0
Scheduling	Priority ("chain")	Page reads before field calls; earlier documents first
Thinking mode	Off (`enable_thinking: false`)	Otherwise reasoning used the whole budget and answers came back empty

7.4 vLLM features used

Feature	How it is used
OpenAI-compatible server	Same request format as llama-server, so engines can be swapped without code changes
Continuous batching	Up to 32 concurrent requests share the GPU
AWQ 4-bit quantised weights	9B vision model fits in 24 GB with room to spare
Multimodal (image) input + pixel cap	Page images scaled to ≤4.2 MP before encoding
Chunked prefill / token budget	Long prompts taken in 2,048-token steps
Priority scheduling	Each request carries a priority number from the app
Streaming responses	Lets the loop guard stop a looping reply early
JSON response mode	Machine-readable fields and audit replies
Chat-template control	Turns off Qwen3.5's internal thinking

8 Why vLLM: methods compared

Every option was measured on the same mixed test documents.

8.1 How latency came down

Latency came down in stages. First, printed pages moved off the GPU to OCR. Then the engine moved from Ollama and llama-server to vLLM, and vLLM was tuned for this model and this GPU. Overall, a mixed document now takes roughly a third of the time it took with the first version. The gains came from keeping the GPU busy and sending it fewer tokens. The things that did not work were blocked mainly by the Turing GPU, its memory, or vLLM's support for this model.

vLLM changes that worked

Change	What it does	Effect
Continuous batching (up to 32 requests)	New requests join the running batch at once instead of waiting for a free slot	The GPU is never idle between calls; the largest single gain over llama-server
AWQ 4-bit weights	Compressed model weights	The 9B vision model fits in 24 GB and leaves room for many requests in parallel
Image size cap	Page images limited to about 4.2 MP, diagram crops to 1 MP	Much shorter prompts; full-size images made page reads roughly twice as slow
Chunked prefill (2,048-token steps)	Long prompts are processed in small pieces alongside text generation	Stable mixing of long and short requests without memory spikes
Priority scheduling ("chain")	Page reads go before field calls, and each page's follow-up calls before new work	Shorter tail at the end of a document; about 10% faster
Thinking mode off	Turns off Qwen3.5's internal reasoning through the chat template	No wasted tokens, and no empty answers when reasoning used up the budget
Streaming + loop guard	Replies are streamed, and one that starts repeating itself is stopped	No more multi-minute stalls on a looping reply
JSON response mode	The model is held to valid JSON for fields and audits	Reliable machine-readable answers, fewer retries
LoRA adapter serving	The fine-tuned adapter runs on the same server as the base model	A handwriting specialist without a second model in memory

Changes that did not work, and why

Change	What happened	Limitation behind it
Prefix caching	Cache hits worked, then the engine deadlocked after a couple of minutes	vLLM bug with Qwen3.5's hybrid layers (software, not hardware)
FlashInfer attention (vLLM v0.23)	Crashed on the first page prompt	Turing has too little on-chip shared memory for these kernels
FlashAttention 2	Not available	Needs compute capability 8.0 (Ampere) or newer
bf16 / FP8 precision	Not available; fp16 only	Turing has no bf16 or FP8 support
Larger prompt batches (16,384 tokens)	Engine ran out of memory mid-run and had to restart	24 GB of GPU memory
More parallel workers / prefetching pages	No gain	The single GPU is already fully used
Qwen3-VL-8B model	About 3× slower on long prompts; memory cache filled up	Without fast attention kernels on Turing, long prompts are slow

8.2 Serving engines compared

Criterion	Ollama	llama-server	vLLM (chosen)
Concurrent requests	Effectively one	4 fixed slots	Up to 32, continuous batching
Batched output speed	Low	~57 tokens/s	~76 tokens/s
Model format	GGUF Q4_K_M	GGUF Q4_K_M	AWQ 4-bit
Request priority	No	No	Yes
LoRA adapter serving	Limited	Limited	Yes, including the vision layers
Weak points	Slow	Context per slot = total ÷ slots (caused cut-offs)	Turing kernels slow; prefix cache unusable; output varies with batch size

8.3 Everything we tried

Method	Expected benefit	Outcome	Decision
Ollama	Simple setup	~30 min per bundle	Replaced
llama-server, 4 slots	Parallel requests	757 s; slower than vLLM	Manual alternative
Printed pages to PaddleOCR	Free the GPU	~60% of model work removed from printed pages	Adopted
vLLM with image cap	Faster batching	599 s; uncapped images took 606 s for main reads alone	Adopted
Compact audit + diagram image cap	Fewer tokens	542 s	Adopted
Chain priority	Shorter tail	604 → 555 s	Adopted

Method	Expected benefit	Outcome	Decision
Loop guard + reply caps	No stalls	Looping pages now stop early	Adopted
Fewer, compact field calls	Less output	529 s; key values still to be compared	Open
Prefix caching	~86k fewer prompt tokens per document	Hits worked, then the engine deadlocked about 2 min in	Rejected
vLLM v0.23 + FlashInfer	Faster attention on Turing	Crashed on the first page prompt	Rejected
Qwen3-VL-8B model	Newer, smaller	3× slower on long prompts; broke the orientation check	Rejected
16,384-token batches	More throughput	Engine crash + 2-minute restart	Rejected
More workers / page prefetch	More parallel work	No gain; GPU already saturated	Rejected
Fields from text only	Shorter prompts	1,007 s and worse fields	Rejected
HTML-only main read	Less output	−15 s only; page content changed	Rejected
Log-prob confidence instead of a second read	Drop one call per page	Caught only 52–64% of real misreads	Rejected

9 GPU and infrastructure limits

What the hardware can do, what it cannot, and what is possible next.

9.1 Hardware

Component	Specification	Role
GPU 0	NVIDIA Quadro RTX 6000, Turing generation (sm_75), 24 GB, fp16	Serves the model
GPU 1	Second card on the same server	Planned second replica
CPU	2 × Intel Xeon Gold 6126 (Skylake-SP), 24 cores / 48 threads, 2.6 GHz base, up to 3.7 GHz; 38.5 MB L3 cache; AVX-512; 2 NUMA nodes	Web app, PaddleOCR for printed pages, PDF and image handling, database

9.2 Why the GPU sets the speed

The GPU is fully used for the whole run, and measured document time follows the token count closely:

> **RULE OF THUMB**
>
> `time ≈ prompt tokens ÷ ~1,500/s + output tokens ÷ ~150/s`
>
> One 11-page bundle ≈ 344,000 prompt tokens and 32,000 output tokens. At these rates, the only ways to make it faster are to send fewer tokens or to add GPU capacity.

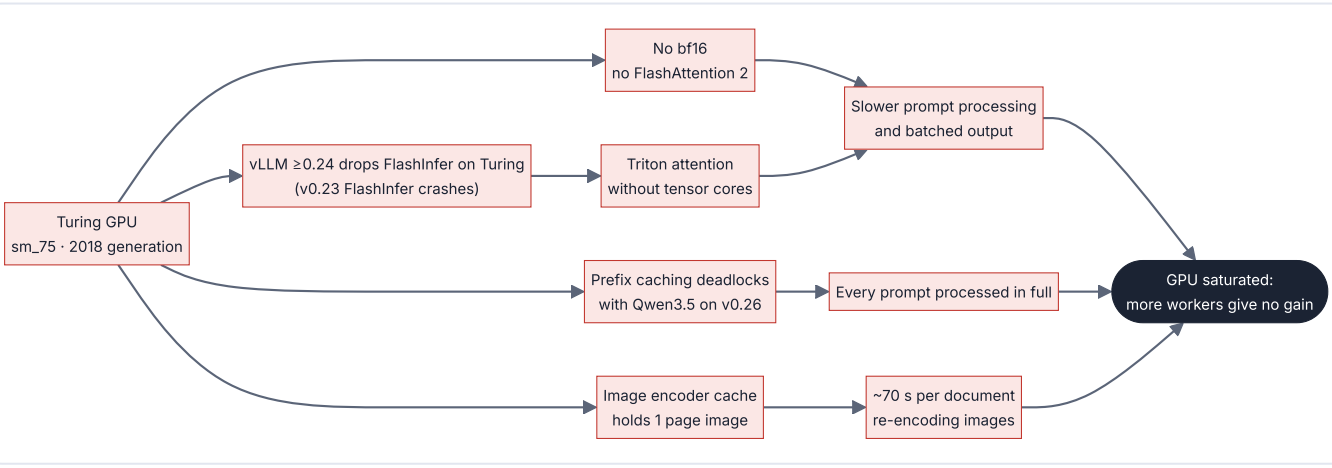


Figure 6: Why the GPU generation limits speed.

9.3 Infrastructure limitations

GPU 0 is the main limit on speed. The Quadro RTX 6000 belongs to NVIDIA's Turing generation (2018), with **compute capability 7.5** (sm_75). Compute capability is the feature level of a GPU, and today's AI serving software is written mainly for 8.0 (Ampere) and above. Most of the speed-ups that make modern model serving fast need those newer features, so on this card vLLM falls back to slower general-purpose code paths.

Speed-ups this GPU cannot use. FlashAttention 2 needs compute capability 8.0 or higher. FlashInfer, the other fast attention library, crashed on this card in vLLM v0.23, because its kernels for this model need more on-chip shared memory than Turing has (64 KB per multiprocessor, against 164 KB on an A100). From vLLM v0.24 it is no longer used on Turing at all. vLLM therefore uses its Triton attention kernels, which on Turing are compiled without the tensor cores. Reported effects are 6–15× slower prompt processing and about 3× slower batched generation than with a fast kernel, with the same accuracy. Qwen3.5's linear-attention layers are Triton kernels too. The card also has no bf16, only fp16, and no FP8, which needs compute capability 8.9 or higher, so the FP8 weight formats that cut memory traffic on newer cards are not available.

Memory bandwidth. Generating text is limited mostly by how fast the GPU can read the model's weights and cache from memory, not by its arithmetic. Every new token needs another pass over the weights. The RTX 6000's GDDR6 memory gives about **672 GB/s**, roughly a third of an A100 (about 2 TB/s) and a fifth of an H100 (about 3.35 TB/s). That sets a hard ceiling on tokens per second that no software setting can raise, which is why output tokens (about 150/s across the whole batch) dominate document time.

Feature	GPU 0 (Quadro RTX 6000)	Newer data-centre GPUs (A100 / L40S / H100)
Compute capability	7.5 (Turing, 2018)	8.0 / 8.9 / 9.0
Memory bandwidth	~672 GB/s (GDDR6)	~2,000 / ~864 / ~3,350 GB/s
On-chip shared memory per multiprocessor	64 KB	164 / 100 / 228 KB
FlashAttention 2 / FlashInfer	No (FlashInfer crashes on v0.23, dropped from v0.24)	Yes
bf16	No (fp16 only)	Yes
Native FP8	No	L40S and H100 only
Attention kernels used by vLLM	Triton, without tensor cores	Optimised tensor-core kernels

The other limitations of the current setup:

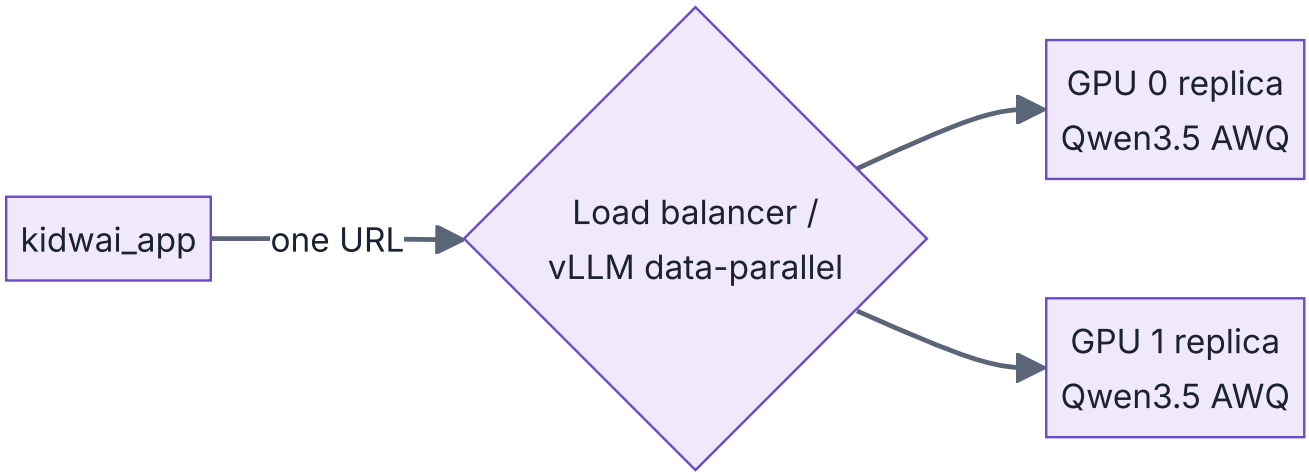
#	Limitation	Impact
L1	Turing GPU: no bf16 or FlashAttention 2; slower attention kernels	Prompt processing several times slower than on newer cards
L2	Prefix caching unusable	~86k extra prompt tokens per document
L3	One GPU, fully used	Tuning has reached its limit
L4	One document at a time	A second upload waits in the queue
L5	Model choice limited	Qwen3.5-9B is the only practical choice on this card
L6	Page images re-encoded several times	~35% of GPU time goes to images
L7	App connects to one model URL	Two GPUs need data-parallel vLLM or a proxy
L8	OCR speed depends on the CPU	~5–30 s per printed page
L9	Training shares GPU 0	Adapter training only off-hours

9.4 What is possible

Today (1 GPU)

- ✅ Printed pages on the CPU, second read and audit on all handwriting
- ✅ Handwritten and mixed pages are processed by GPU
- ✅ Region LoRA adapter as a second reader (shadow mode)
- ✅ Off-hours fine-tuning on GPU 0
- ❌ Results in seconds per document
- ❌ Several documents in parallel (GPU already saturated)
- ❌ Larger or newer vision models (too slow or unstable on sm_75)

With GPU 1



- **Split one document across both:** ~40% lower latency in a teammate's early test.
- **Two documents at once** (`EXTRACT_CONCURRENCY=2`): ~2× throughput, same latency per document untested.
- **One GPU serving, one training:** fine-tuning during the day without slowing users.

Needs new hardware

- A large cut in latency (to ~2–3 min per document) needs FlashAttention or FlashInfer and bf16, which means Ampere-generation GPUs or newer.
- Prefix caching and newer vision models need kernel support Turing lacks in current vLLM.
- Many users uploading at once needs more GPUs or a larger card.

10 Red-word detection

How the system decides which handwritten words a person must check.

The aim is a red word that is **worth a reviewer's time**: a likely misread, especially in a drug name, dose, number or date. The design therefore combines several **independent signals** that raise flags with **filters** that remove noise, and adds **protections** so clinically critical flags are never removed.

> **CORE RULE**
> The system **marks, it never replaces**. Only the first (main) read's text is kept. The other checks only vote on whether a word is doubtful, so a check can never rewrite a clinical value.

10.1 Signals that raise a flag

Signal	How it works	Catches
Self-marked doubt	The main read marks words it is unsure of	Words the model itself could not read clearly
Second independent read	The page is read again at temperature 0.4 and compared with the first read **line by line**. Disagreements are flagged. Edge punctuation is ignored ("7." = "7"), but a different digit always counts ("2.5" ≠ "25").	Unstable readings: where two reads disagree, the handwriting is ambiguous
Word audit	A critic call checks the transcript against the image, focusing on the handwritten lines; spelling candidates are passed as hints	Misreads both reads agreed on
Unknown drug	A word in medicine position (after T., Tab, Cap, Inj, Syp...) that matches no known drug	Garbled drug names, e.g. "Leftazidine"
Dose rule	Implausible doses for known drug strengths, e.g. a three-figure gram dose ("500g")	Unit and digit misreads
Look-alike drugs	Drug pairs from the ISMP List of Confused Drug Names (2024) involving the project's drugs, plus oncology additions	Dangerous mix-ups, e.g. ceftriaxone / ceftazidime
Word-shape checks	No vowels ("Bcm"), not in any English, medical, drug or shorthand list, or a 2–4 letter capitals code no list knows	"Letter soup" from illegible writing

11 Field extraction and patient records

From page text to structured, per-patient clinical data.

11.1 Field extraction

One field-extraction process runs per document, alongside the page reading. Long documents are split into **page chunks** that run concurrently. Their results are merged, with fields de-duplicated by key and medicines by name, so every page contributes. For pages with substantial handwriting, the call also receives that page's finished transcript, which recovers rows a single reading of the image would miss. Output is JSON: identifiers, demographics, diagnosis, medicines (name, dose, route, frequency, duration), lab values, dates and doctors.

11.2 Patient identity: UHID only

> **IDENTITY RULE**
>
> A document is linked to a patient **only by UHID**. Names repeat and vary between documents, and hospital or registration numbers are per visit or legacy, so neither may decide that two documents belong to the same person. A document with no readable UHID, an uncertain one, or two different ones stays **unlinked**, with the reason recorded, for a person to resolve.

Names are still used to *find* a patient. Fuzzy name search (PostgreSQL `pg_trgm`) ranks candidates for a person to pick from, but never links anything by itself.

11.3 Routing into typed tables

Each extracted value becomes a staging row (one row per value, with its page and date). Rule-based routing then places each row into a **typed table**, with deterministic rules first. Every staging row records where it went (`routed_to`) or that no rule claimed it, so nothing is lost without a trace.

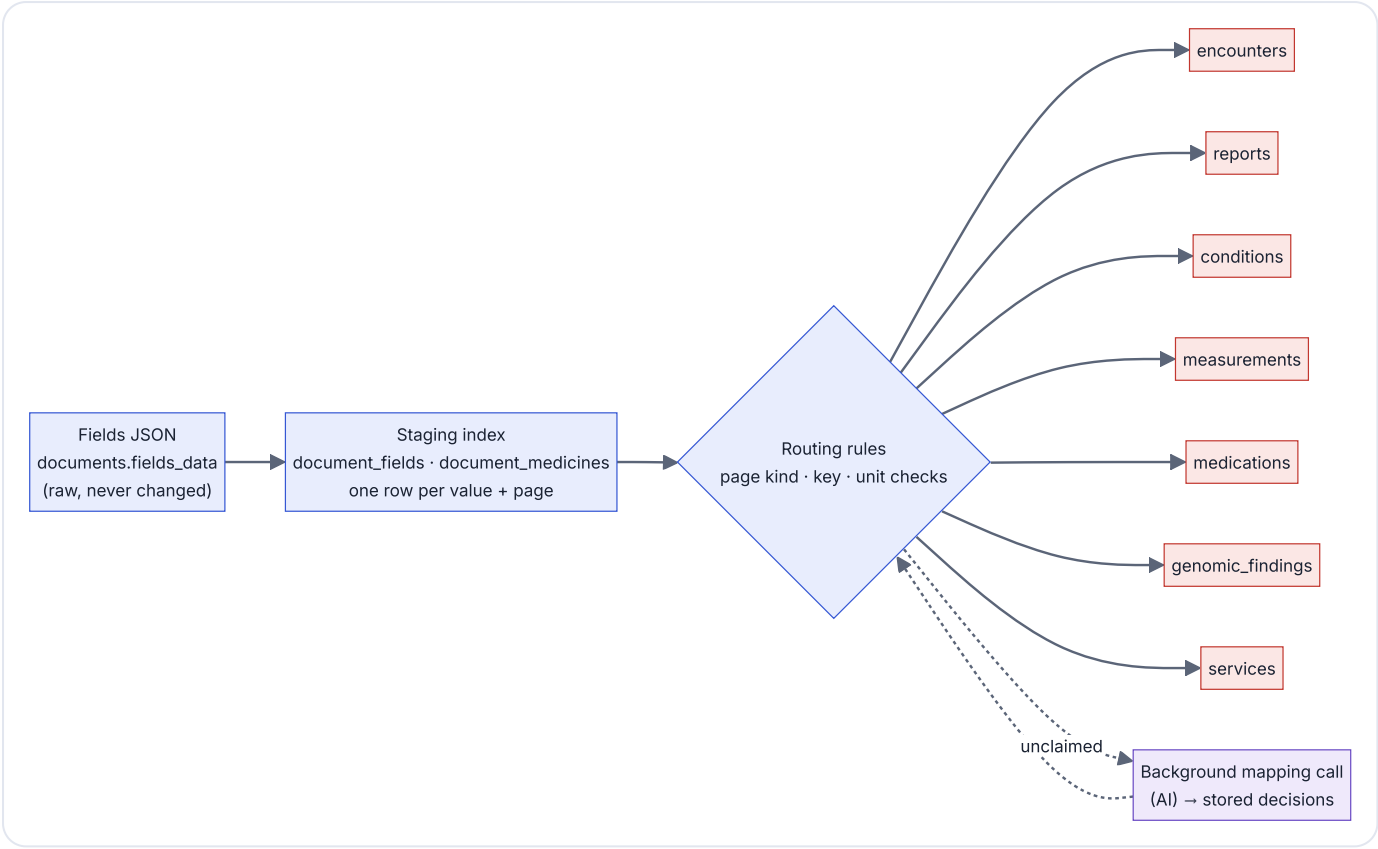


Figure 7: From extracted fields to typed patient tables.

11.4 Background mapping

Keys the rules don't recognise are sent to the model in a **background mapping call** after the document is saved, so the user never waits for it. The model classifies the page and proposes a target table for each unknown key. Decisions are stored (`field_mappings` , `page_classifications`) and reused for future documents. Only trusted decisions are applied, and each one's effect is recorded per document (`mapping_outcomes`): rows placed, and rows a check refused.

12 Data management

Where data lives, how it is structured, and the rules that keep it consistent.

12.1 Data stores

Store	Technology	Holds
Main database	PostgreSQL 16 (`kidwai_users_db`)	Users and SSO accounts, documents and their extracted data, patients, staging index, typed clinical tables, mapping decisions
File storage	Azure Blob API via Azurite (`kidwai_azurite`)	Original uploaded files, corrected PDFs

12.2 Table reference

Accounts and documents

Table	Purpose
`users`	Local and SSO accounts; PBKDF2 password hashes for local accounts; constraints on the sign-in source
`documents`	One row per upload: owner, patient link and match method, blob name, raw fields and medicines (JSONB), pages snapshot, corrected pages

Patient index

Table	Purpose
`patients`	One row per UHID per account; demographics and every identifier seen, rebuilt from the patient's documents
`document_fields`	Staging: one row per extracted value, with page, date, canonical key, row index and where it was routed
`document_medicines`	Staging: one row per prescription line

Typed clinical tables

Table	FHIR / OMOP analogue	Holds
`encounters`	Encounter	Visits: date, clinical text, facility, doctor registration
`reports`	DiagnosticReport	Lab and diagnostic reports, including who signed them
`conditions`	Condition	Diagnoses
`measurements`	Observation / OMOP measurement	All lab values and vitals in one table
`medications`	MedicationRequest	Prescribed drugs, dose, route, frequency, duration
`genomic_findings`	Genomics variant observation	Genetic test results
`services`	(service / procedure)	Services and procedures

Each typed table has a `metadata` JSONB column for anything its columns don't hold, so a fixed schema can still keep every detail.

Mapping decisions

Table	Purpose
`field_mappings`	Stored decisions from the mapping call: unknown key → target table/column
`page_classifications`	Stored page-kind decisions
`mapping_outcomes`	Per document: rows each decision placed, rows a check refused

13 Fine-tuned adapter (LoRA)

A handwriting specialist trained on the hospital's own documents.

13.1 What the adapter is

A **LoRA adapter** is a small set of extra weights trained on top of the base model. The base model, Qwen3.5-9B, stays unchanged. At request time, vLLM applies the adapter only to the requests that ask for it, so one GPU serves both the general model and the specialist.

The **region adapter** ("region-v2") is a crop reader. It is given a small image of one handwritten region, such as a line or a short block, and returns the text in it. It was trained for exactly that task, so it reads handwriting better than the base model, but it cannot read a whole page or produce the page layout.

13.2 How it was trained

Item	Value
Method	QLoRA (4-bit nf4 base during training), rank 16
Base model	Qwen/Qwen3.5-9B
Layers adapted	248 language-model modules and 110 vision-encoder modules
Training data	Cropped handwritten regions from the hospital's own documents, with checked text
Best checkpoint	Step 500
Hardware	GPU 0, trained off-hours
Evaluation	145 regions from documents held out of training, so no page was seen twice

Adapting the vision layers as well as the language layers matters, because the gain comes partly from **seeing** doctors' handwriting better, not only from guessing words better.

13.3 Results on vLLM

Measured on the same 145 held-out regions, served by vLLM v0.26 exactly as production would run it:

Metric	Base model	With adapter
Character error rate (lower is better)	52.1%	**18.7%** (about 64% fewer wrong characters)
Exact region match	5.5%	**18.6%**
Numbers read correctly	60.8%	**73.3%**

13.4 Why adapter requests are slower

Each one of the adapted layers does an extra small calculation for every token: the base layer's output plus the adapter's correction. Generating text one token at a time is limited by kernel launches and memory reads rather than by maths, so these extra steps roughly double the time per token.

vLLM processes all requests in flight as one batch. When even one adapter request is in the batch, the whole batch takes the LoRA path, which is why normal requests slow down too.

> **IS IT BECAUSE OF THE GPU?**
> Partly. LoRA adds work on any GPU, because each adapted layer computes an extra correction for every token. On this Turing GPU the effect is larger, because vLLM's LoRA kernels cannot use the fast tensor-core path on this generation. A newer GPU (Ampere or later) would reduce the gap, but not remove it. How much smaller it would be has not been measured.

Options to make it faster: merge the adapter into the model weights (no per-token overhead, but then that copy is always the specialist), adapt fewer layers (needs retraining, may lose accuracy), or move to a newer GPU.

14 Pros and cons

An overall assessment of the product as delivered.

✓ Pros

- **Fully on-premise:** no patient data leaves the hospital; no per-page cloud costs.
- **Reads handwriting**, which generic OCR tools cannot do reliably.
- **Safety by design:** uncertain words are flagged, never silently changed; drug, dose and look-alike checks.
- **Review effort cut sharply:** 71% fewer red words.
- **3.4× faster** than the first version, on the same GPU.
- **Smart use of hardware:** printed pages on the CPU; the GPU only for handwriting.
- **Structured, standards-shaped data:** typed tables aligned with FHIR R4 / OMOP; UHID-only identity.
- **Strong data integrity:** raw kept, derived rebuilt, account isolation enforced by the database.
- **Robust:** one failed page doesn't fail a document; loop guard; bounded calls.
- **Open, swappable stack:** OpenAI-compatible interface; no vendor lock-in.

✗ Cons

- **Speed limited by an older GPU:** modern speed-ups are not available on the Turing generation.
- **One document at a time** on a single GPU.
- **Human review is still required** for red words; red is not exhaustive.
- **Small run-to-run variation** in output and in field counts.
- **No end-to-end accuracy figure yet** on a reference set.
- **Adapter** trained on few writers; not yet live.
- **Single server:** no high availability; a GPU fault stops processing.

A Glossary

Terms used in this document.

Term	Meaning
AWQ	A 4-bit compression method for model weights; makes a large model fit a smaller GPU
CER	Character error rate: the share of characters read wrongly (lower is better)
FlashAttention / FlashInfer	Fast attention implementations that need newer GPUs
LoRA / QLoRA	A small trainable add-on to a model; QLoRA trains it on a compressed base model
OCR	Optical character recognition: reading printed text from images
Probe	A short, targeted model question (e.g. "is this page upright?")
Red word	A word flagged as uncertain, shown in red for a person to confirm
Token	A piece of text or image the model processes; time and cost scale with tokens
Turing (sm_75)	NVIDIA's 2018 GPU generation, which includes the Quadro RTX 6000
UHID	Unique Hospital ID: the only identifier used to link documents to a patient
vLLM	An open-source high-throughput server for running AI models